Übung 09 – Rechnen und Gruppieren

1000 Datenbank · Teil 3 SQL · ca. 45 Minuten · MySQL Workbench



Nonna zählt nach

„Wie viele Rezepte habe ich? Wie gut kommen sie an? Und wer von meinen Lesern schreibt am meisten?“

Ziel	Mit Aggregatfunktionen rechnen, mit GROUP BY pro Gruppe auswerten und Gruppen mit HAVING filtern – wie auf den Folien an Marios Datenbank.
Startpunkt	`nonnas_blog.sql` (Ordner `material`) – legt die Datenbank `nonnas_blog` an; **frisch ausführen**, damit alle dieselben Daten haben
Tabellen	`kategorie` · `rezept` (rezept_id, titel, dauer_min, schwierigkeit, veroeffentlicht, kategorie_id) · `zutat` (zutat_id, name, einheit) · `rezept_zutat` (rezept_id, zutat_id, menge) · `leser` (leser_id, vorname, nachname, email) · `kommentar` (kommentar_id, rezept_id, leser_id, text, sterne, geschrieben)
Ablage	deine Lösungen in `datenbank/abfragen_nonna.sql` (mit Kommentar `-- Übung 09, Aufgabe …`)
Folien	Präsentation „1000 Datenbank“ – vor der Übung erklärt und vorgemacht; bei jeder Aufgabe steht die passende Folie

Aufgabe 1 – Zählen, kleinster und größter Wert

Hilfe: *Präsentation „1000 Datenbank“, Folie 0670 (Nummer unten rechts)*

- a) Wie viele Rezepte gibt es?

```
USE nonnas_blog;
SELECT COUNT(*) AS anzahl_rezepte
FROM rezept;
```

Es gibt 6 Rezepte. COUNT(*) zählt alle Zeilen der Tabelle rezept.

- b) Wie lange dauert das kürzeste, wie lange das längste Rezept?

```
SELECT MIN(dauer_min) AS kuerzeste_dauer,
       MAX(dauer_min) AS laengste_dauer
FROM rezept;
```

Die kürzeste Dauer beträgt 15 Minuten, die längste 90 Minuten. MIN und MAX ermitteln den kleinsten und größten Wert.

Aufgabe 2 – Durchschnitt und Summe

Hilfe: *Präsentation „1000 Datenbank“, Folie 0670 (Nummer unten rechts)*

- a) Wie viele Sterne geben die Leser im Durchschnitt? (auf 1 Stelle gerundet)

```
SELECT ROUND(AVG(sterne), 1) AS durchschnitt_sterne
FROM kommentar;
```

Die sieben Kommentare ergeben im Mittel 4,4 Sterne. ROUND rundet den Durchschnitt auf eine Nachkommastelle.

- b) Wie viel Parmesan (zutat_id 10) braucht man für alle Rezepte zusammen?

```
SELECT SUM(menge) AS parmesan_gramm
FROM rezept_zutat
WHERE zutat_id = 10;
```

Insgesamt braucht man 190 g Parmesan. WHERE wählt die Zeilen mit zutat_id 10 aus. SUM addiert die Mengen 100, 30 und 60 g.

Aufgabe 3 – Pro Gruppe

Hilfe: *Präsentation „1000 Datenbank“, Folie 0680, 0700 (Nummer unten rechts)*

Gehe vor wie auf der Folie „GROUP BY in drei Schritten“ (wonach gruppieren, was berechnen, welche Gruppen behalten).

- a) Wie viele Rezepte gibt es pro Kategorie?

```
SELECT kategorie_id, COUNT(*) AS anzahl_rezepte
FROM rezept
GROUP BY kategorie_id
ORDER BY kategorie_id;
```

Es gibt je 2 Rezepte in den Kategorien 1 (Vorspeise), 2 (Hauptgericht) und 3 (Dessert). GROUP BY bildet eine Gruppe pro Kategorie, und COUNT(*) zählt die Rezepte darin.

- b) Wie viele Kommentare hat jeder Leser geschrieben?

```
SELECT l.leser_id, COUNT(k.kommentar_id) AS anzahl_kommentare
FROM leser AS l
LEFT JOIN kommentar AS k ON k.leser_id = l.leser_id
GROUP BY l.leser_id
ORDER BY l.leser_id;
```

Leser 1, 2 und 4 haben je 2 Kommentare, Leser 3 hat 1 und Leser 5 hat 0. LEFT JOIN behält auch Leser ohne Kommentar. COUNT(k.kommentar_id) zählt nur vorhandene Kommentar-IDs.

Aufgabe 4 – Gruppen filtern

Hilfe: *Präsentation „1000 Datenbank“, Folie 0690 (Nummer unten rechts)*

Nur die Leser, die mehr als einen Kommentar geschrieben haben. Warum geht das nicht mit WHERE?

```
SELECT l.leser_id, COUNT(k.kommentar_id) AS anzahl_kommentare
FROM leser AS l
LEFT JOIN kommentar AS k ON k.leser_id = l.leser_id
GROUP BY l.leser_id
HAVING COUNT(k.kommentar_id) > 1
ORDER BY l.leser_id;
```

Die Leser 1, 2 und 4 haben je 2 Kommentare. WHERE filtert einzelne Zeilen vor der Gruppierung. HAVING prüft die fertigen Gruppen, nachdem COUNT die Kommentare pro Leser gezählt hat.

Das kann ich jetzt

- ☐ Ich zähle, summiere und bilde Durchschnitte.
- ☐ Ich gruppiere mit GROUP BY und kenne die Regel für das SELECT.
- ☐ Ich unterscheide WHERE und HAVING.

Zusatzaufgabe

Welche Rezepte haben mindestens 4 Zutaten?

```
SELECT r.titel, COUNT(*) AS anzahl_zutaten
FROM rezept AS r
JOIN rezept_zutat AS rz ON rz.rezept_id = r.rezept_id
GROUP BY r.rezept_id, r.titel
HAVING COUNT(*) >= 4
ORDER BY r.rezept_id;
```

Tiramisu hat 6 Zutaten. Lasagne, Minestrone und Risotto ai Funghi haben jeweils 4. Die Gruppierung nach Rezept-ID hält Rezepte mit gleichem Titel getrennt.

Zum Schluss: sichern mit Git

Speichere alles (Strg + S) und sichere deinen Stand im Repository `marios-pizza`:

```
git add .
git commit -m "SQL: Rechnen und Gruppieren mit Nonna's Blog (Übung 09)"
git push
```

Kontrolle: auf github.com das Repository neu laden – dein Commit und die neuen Dateien sind zu sehen.